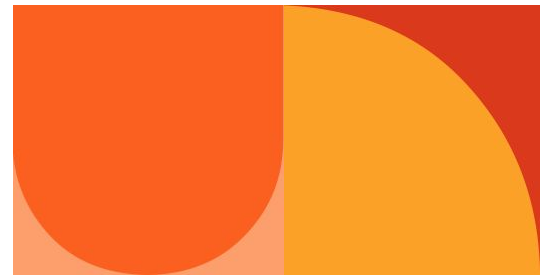
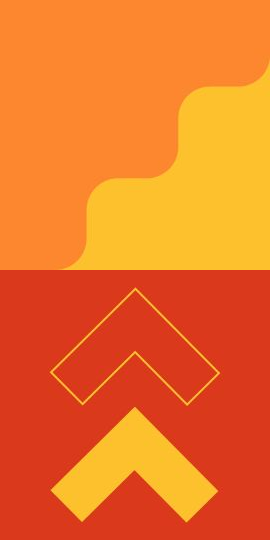




Sistemas Digitais

Aula 10: Comunicação Serial



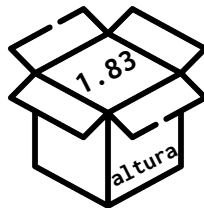


Nos capítulos anteriores de Sistemas Digitais...

Aprendemos sobre:

- Variáveis;
- Tipos de variáveis;
- Push Button;
- Estruturas de Decisão (**if** e **else**);
- Operadores Relacionais;
- Entrada Digital (**digitalRead(pino)**).

Variáveis e seus tipos



Variáveis são espaços de memória nomeados onde podemos armazenar valores de vários tipos:

Numéricos:

int - números inteiros (1, 2, 3, -8, -50)

float - números reais (1.0, 5.5, -32.45)

Lógicos:

bool - valor lógico de verdadeiro (**true**) ou falso (**false**)

Texto:

char - representação de um caractere ('a', 'd', 'x', '?', '7')

String - sequência de caracteres ("Eita", "0xe", "Vish")

Mais sobre os tipos de dados em Arduino [aqui](#).

Push Button

O botão é um interruptor momentâneo e pode ser usado de duas formas:

- 1 - Como um interruptor elétrico;
- 2 - Como um sensor digital;



Not Pressed



Pressed

Estruturas de Decisão

Em um sketch de Arduino podemos escrever códigos que só são executados se uma determinada condição for atendida. Essas condições são comparações lógicas (booleanas) que resultam sempre em **verdadeiro** (true) ou **falso** (false).

```
1 int idade = 10;
2
3 void setup() {
4   if (idade > 18) {
5     // abrir porta
6   } else {
7     // fechar porta
8   }
9 }
```

Entrada Digital

Para receber alguma informação de um pino digital, é possível usar o comando:

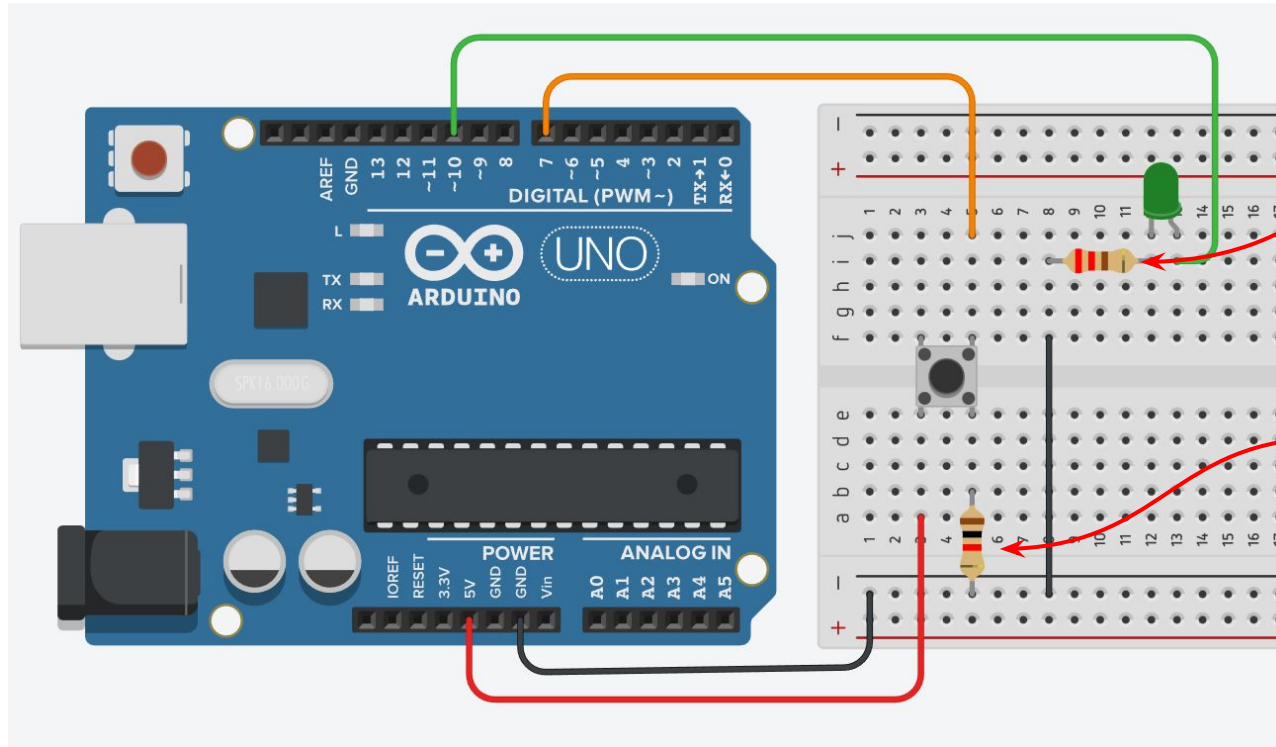
```
estadoBotao = digitalRead(buttonPin);
```

*Variável que irá receber o
valor lido (HIGH ou LOW)*

*Pino que o
Arduino irá ler*

**⚠ Para funcionar corretamente, é necessário que o pino lido
seja configurado como INPUT**

Push Button como sensor



Resistor de 220KΩ

Resistor de 1KΩ

Push Button como sensor

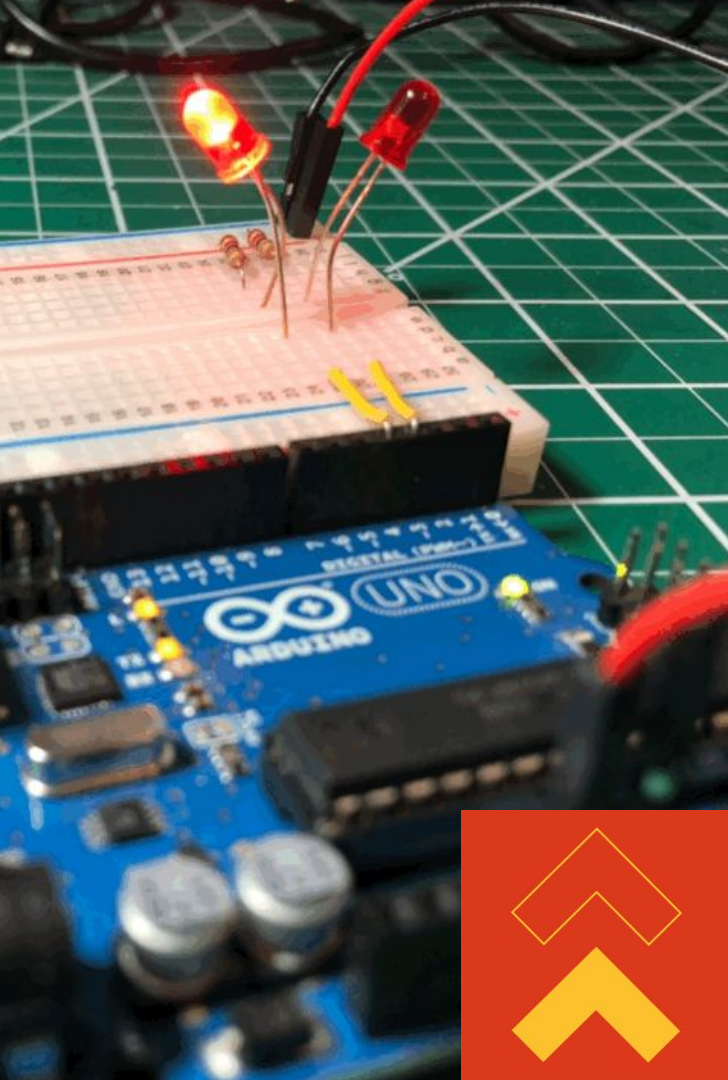
```
1  const int LED_PIN = 10;
2  const int BTN_PIN = 7;
3  bool state;
4
5  void setup() {
6      pinMode(LED_PIN, OUTPUT);
7      pinMode(BTN_PIN, INPUT);
8  }
9
10
11 void loop() {
12     state = digitalRead(BTN_PIN);
13
14     if(state == HIGH) {
15         digitalWrite(LED_PIN, HIGH);
16     } else {
17         digitalWrite(LED_PIN, LOW);
18     }
19 }
```

A variável state poderá ter os valores HIGH ou LOW

Como o botão é um sensor, ele é definido como INPUT

O valor de state é atualizado a cada repetição do loop()

A LED só liga se o estado do botão for HIGH (pressionado)



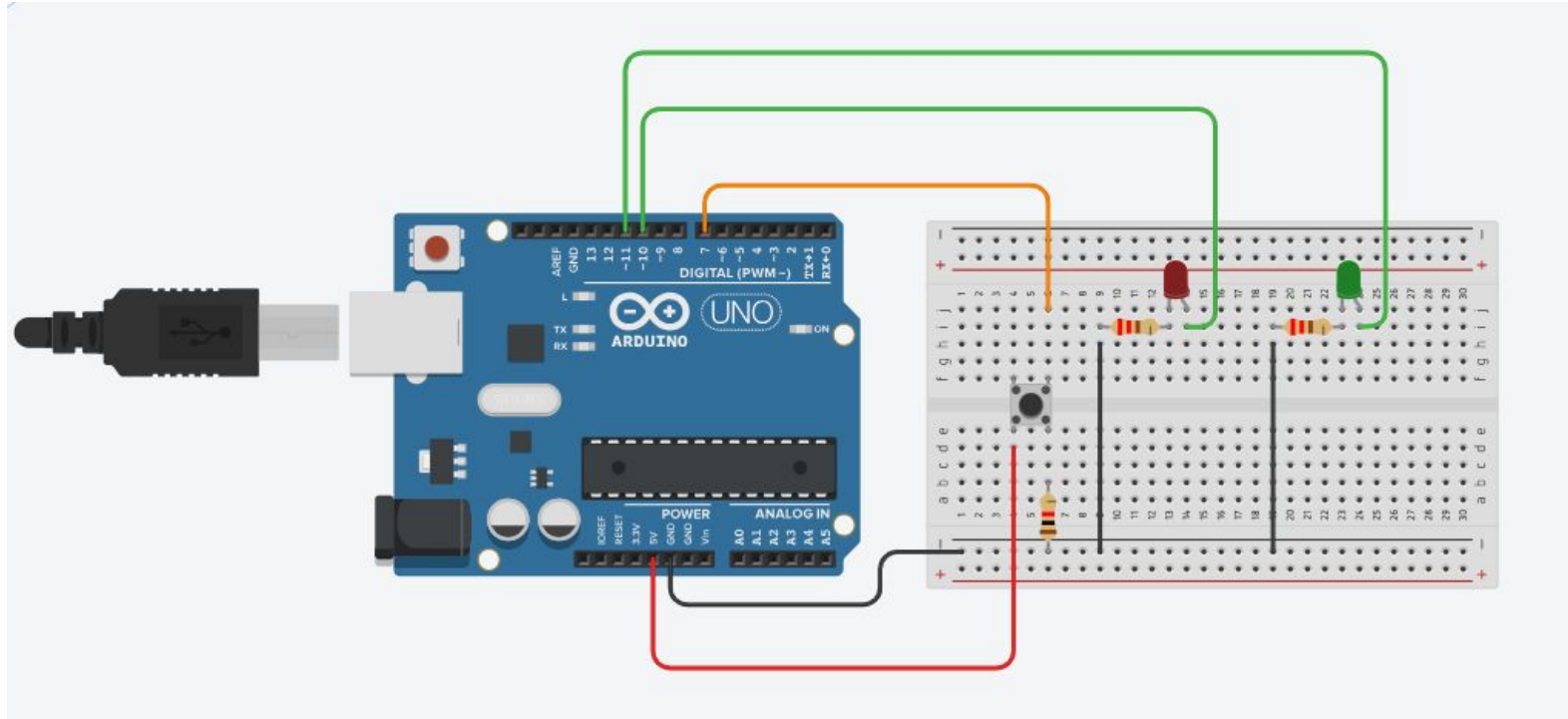
Desafio

Adicione mais uma LED ao circuito.

Quando o botão for pressionado, uma das LEDs deve acender, quando o botão for solto, outra LED deve acender.



Push Button com duas Leds





Push Button com dois Leds

```
1 const int LED_PIN_RED = 10;
2 const int LED_PIN_GREEN = 11;
3 const int BTN_PIN = 7;
4 bool state;
5
6 void setup() {
7   pinMode(LED_PIN_RED, OUTPUT);
8   pinMode(LED_PIN_GREEN, OUTPUT);
9   pinMode(BTN_PIN, INPUT);
10 }
```

```
1 void loop() {
2   state = digitalRead(BTN_PIN);
3
4   if(state == HIGH) {
5     digitalWrite(LED_PIN_RED, HIGH);
6     digitalWrite(LED_PIN_GREEN, LOW);
7   } else {
8     digitalWrite(LED_PIN_RED, LOW);
9     digitalWrite(LED_PIN_GREEN, HIGH);
10  }
11 }
```



Comunicação Serial

Serialização de Dados



Quando precisamos receber e enviar dados entre diferentes dispositivos digitais, podemos usar a comunicação serial entre eles.

A comunicação serial é um método de transferência de dados que envia um bit de cada vez, em série, através de um único canal de comunicação entre o transmissor e o receptor.



Serialização de Dados



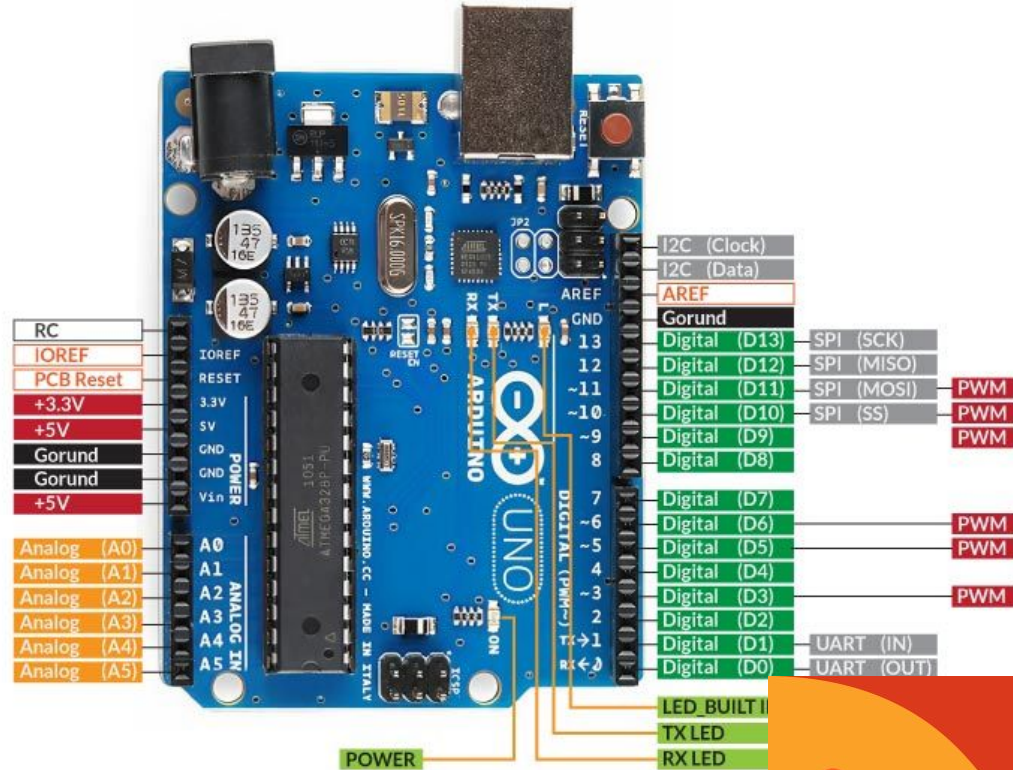
Dentre os vários exemplos de conexões de comunicação serial em eletrônicos e sistemas, se destacam:

- Comunicação UART em nível TTL (5V)
- USB (Programação e monitor serial)
- I2C (Inter-Integrated Circuit) - (Pinos A4/A5)
- SPI (Serial Peripheral Interface) (Pinos 10 até 13.



Comunicação Serial no Arduino

As duas principais portas de comunicação serial do Arduino é a porta USB e os pinos 0 e 1 (RX e TX). Para usar a comunicação serial com o Arduino, no código, basta inicializar a comunicação e enviar as informações com as funções print.





Comunicação Serial no Arduino

`Serial.begin(speed)`

Inicializa a comunicação serial, configurando a taxa de transferência em bits por segundo (*baud rate*) para transmissão serial.

`Serial.print(val)`

Imprime dados na porta serial como texto.

`Serial.println(val)`

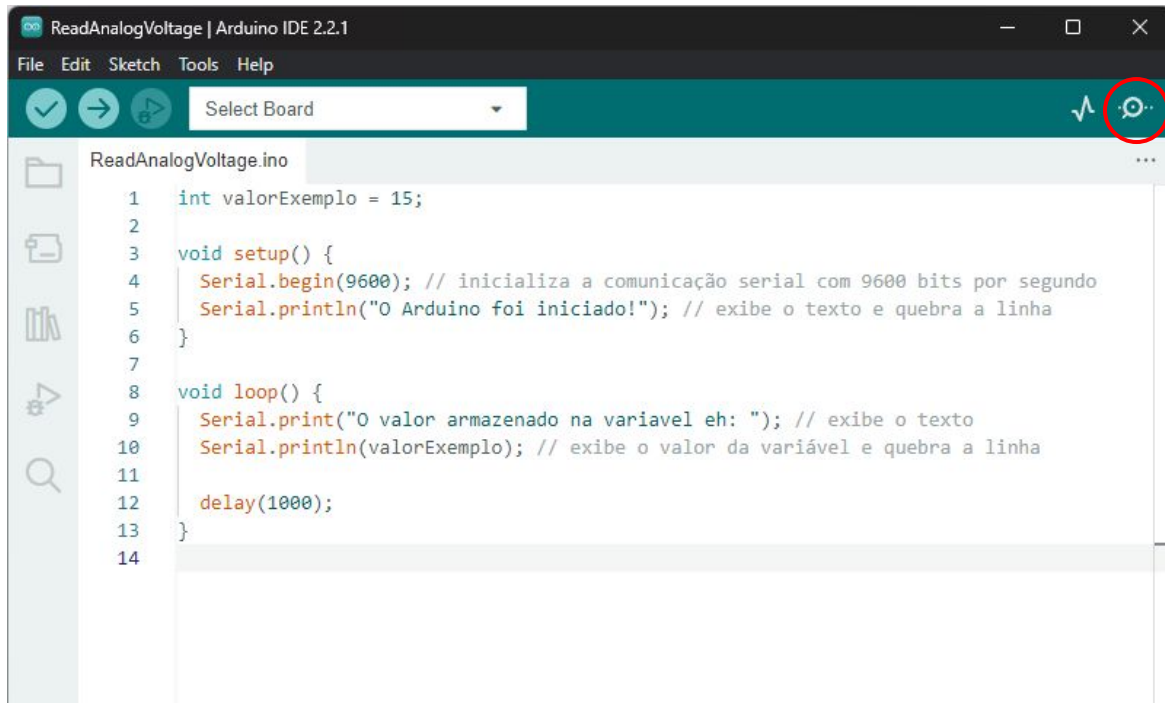
Imprime dados na porta serial como texto, seguido por uma nova linha ('\n').



Exibindo o valor de uma variável pela Serial

```
1 int valorExemplo = 15;
2
3 void setup() {
4   Serial.begin(9600); // inicializa a comunicação serial com 9600 bits por segundo
5   Serial.println("O Arduino foi iniciado!"); // exibe o texto e quebra a linha
6 }
7
8 void loop() {
9   Serial.print("O valor armazenado na variavel eh: "); // exibe o texto
10  Serial.println(valorExemplo); // exibe o valor da variável e quebra a linha
11
12  delay(1000);
13 }
```

Abrir o Monitor Serial na IDE do Arduino



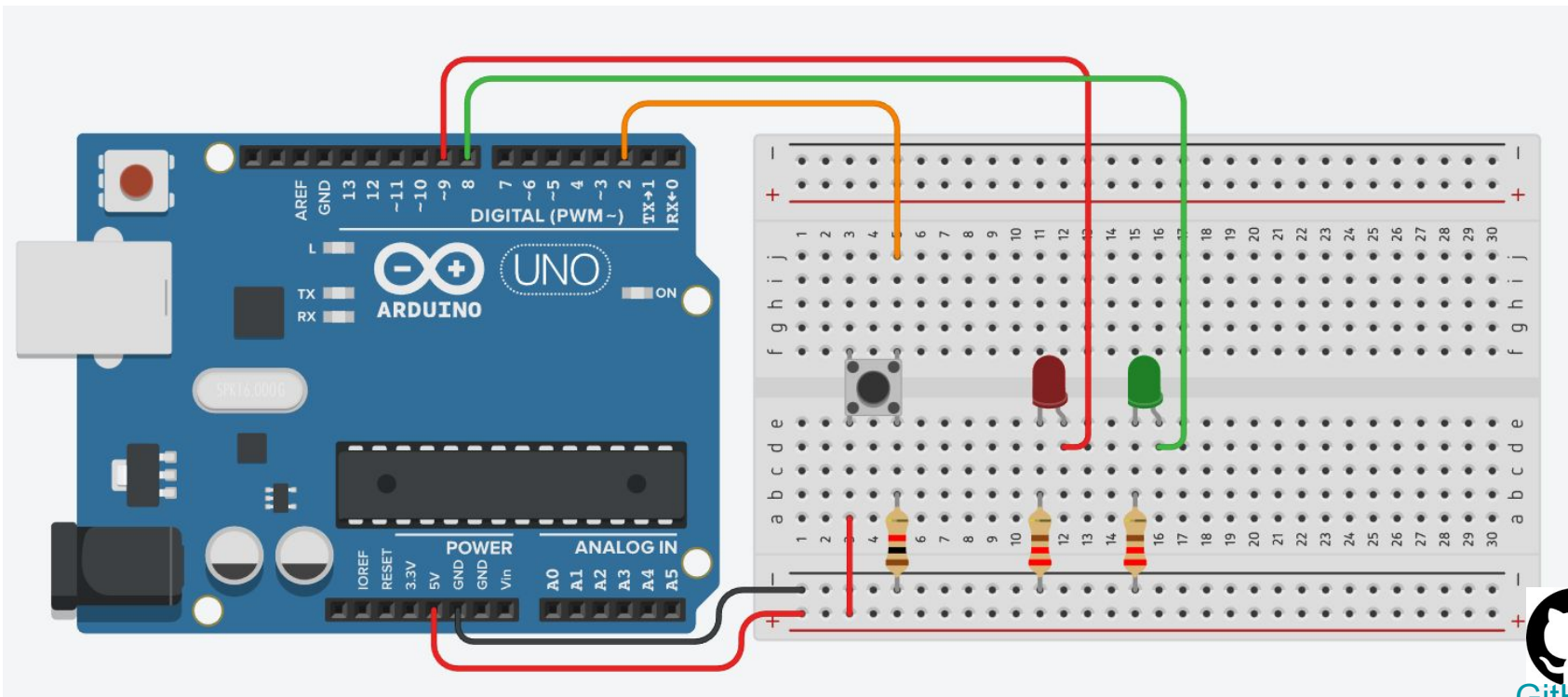
*Clique na lupa para
abrir o Monitor Serial*



Exercício

**Informar se o botão está
pressionado ou não pela Serial.**

Informar se o botão está pressionado ou não pela Serial.





Exercício:

Informar se o botão está pressionado ou não pela Serial.

```
1  int vermPin = 9;
2  int verdPin = 8;
3  int btnPin = 2;
4  bool btnStatus;
5
6  void setup() {
7      Serial.begin(9600);
8
9      pinMode(vermPin, OUTPUT);
10     pinMode(verdPin, OUTPUT);
11     pinMode(btnPin, INPUT);
12 }
13 // ...
```

```
14 void loop() {
15     btnStatus = digitalRead(btnPin);
16
17     if(btnStatus == HIGH) {
18         digitalWrite(vermPin, HIGH);
19         digitalWrite(verdPin, LOW);
20         Serial.println("Botao pressionado!");
21     } else {
22         digitalWrite(vermPin, LOW);
23         digitalWrite(verdPin, HIGH);
24         Serial.println("Botao solto!");
25     }
26 }
```



Leitura Serial





Leitura de Dados Seriais

Além de enviar dados, também é possível receber dados pela porta Serial. Para isso, usamos os seguintes comandos:

`Serial.available()`

Retorna o número de bytes (caracteres) disponíveis para leitura da porta serial.

É usado para verificar se dados foram recebidos.

`Serial.read()`

Lê dados recebidos na porta serial, retornando um byte (char) por vez.

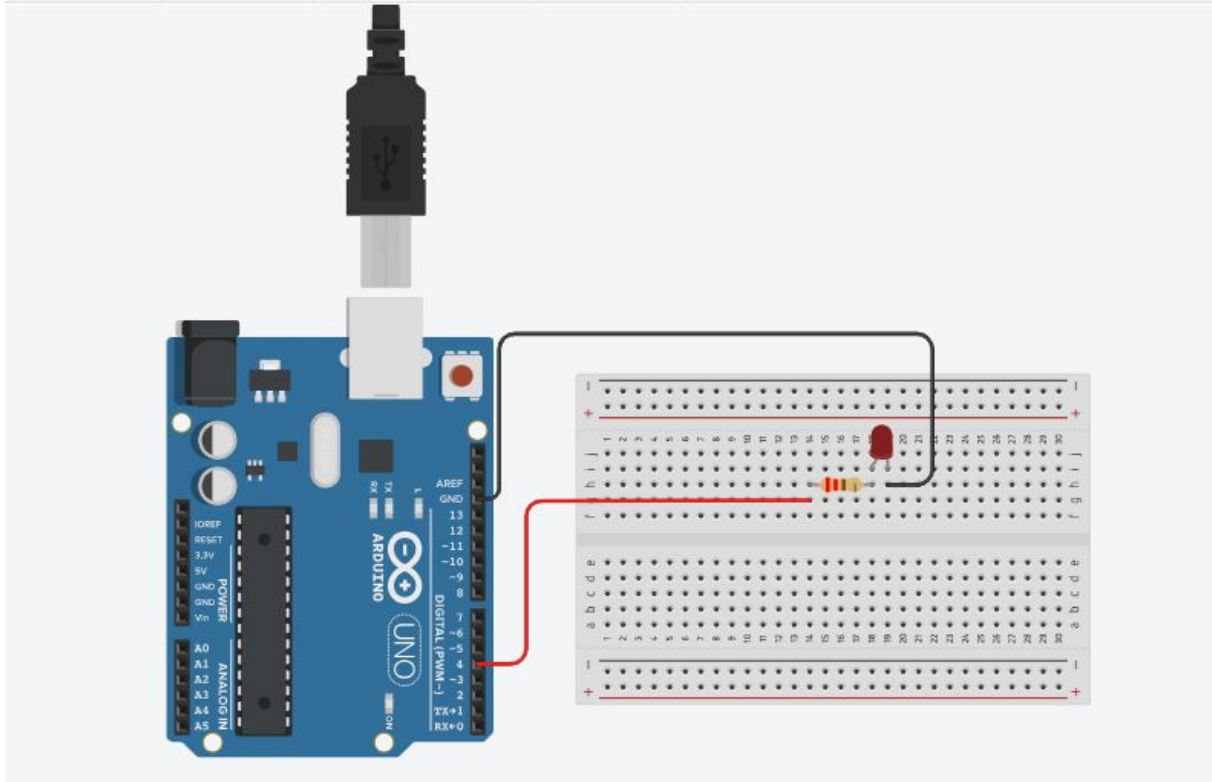




Exercício

Se o caracter 'l' for recebido pela Serial, acende a LED, se o caractere 'd' for recebido, apaga a LED

Ligando e apagando a LED via Serial



Ligando e apagando a LED via Serial

```
1  int ledPin = 4;
2  char input;
3
4  void setup() {
5      Serial.begin(9600);
6      pinMode(ledPin, OUTPUT);
7  }
8
9  void loop() {
10     if(Serial.available() > 0) {
11         input = Serial.read();
12
13         if(input == 'l'){
14             digitalWrite(ledPin, HIGH);
15         } else if (input == 'd') {
16             digitalWrite(ledPin, LOW);
17         }
18     }
19 }
```

Variável que vai receber o valor da Serial

Inicializa a comunicação Serial com 9600 bits por segundo

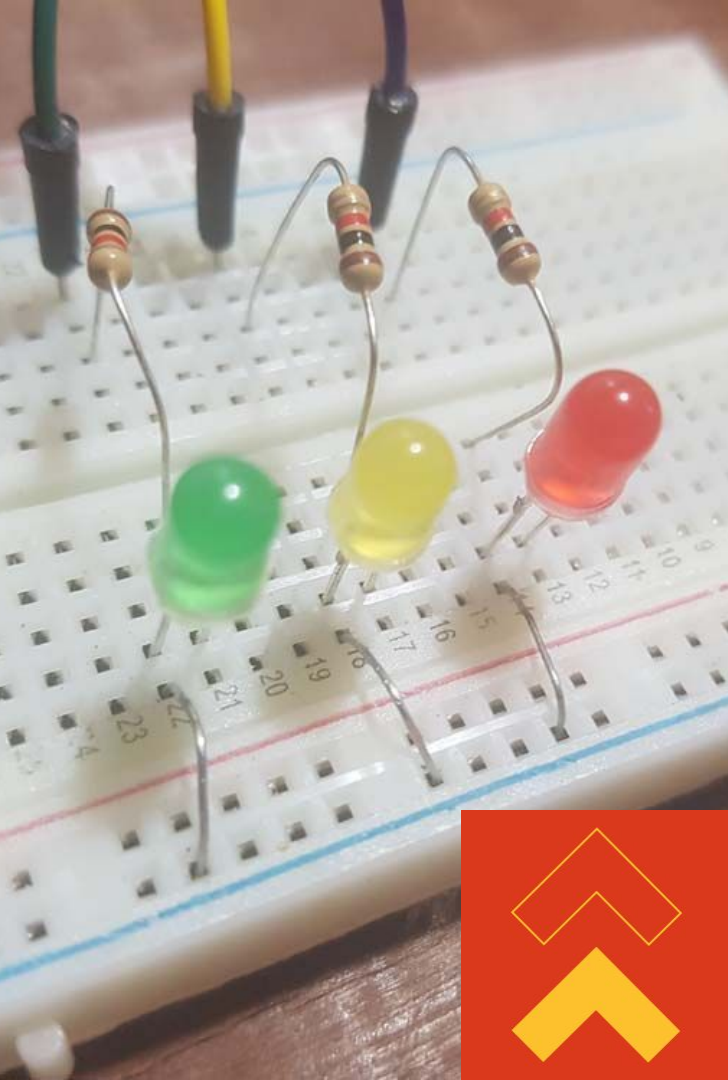
Verifica se têm dados disponíveis

Lê o caracter da Serial e armazena na variável input

Avalia qual o caractere para acender ou apagar a LED



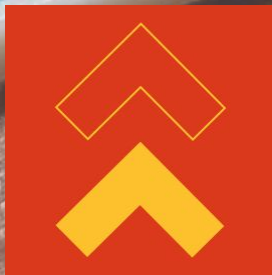
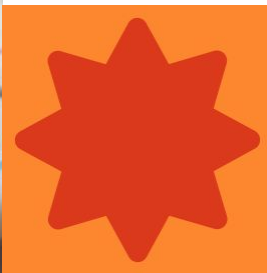
Desafio: Semáforo Serial



Desafio: Semáforo Serial

Monte um circuito com 3 LEDs (vermelha, amarela e verde) que sejam controladas por informações enviadas pela Serial, seguindo a tabela:

Dado	Comportamento
v	Acende a LED Vermelha
o	Apaga a LED Vermelha
a	Acende a LED Amarela
u	Apaga a LED Amarela
d	Acenda a LED Verde
i	Apaga a LED verde



Dica: Use a estrutura condicional Switch-Case!



Alguma pergunta?

